Printout

Thursday, March 19, 2026 11:26 AM

Lab - 1 | 22 January 2026

↳ 6 cells hidden

Lab - 2 | 29 January 2026

↳ 14 cells hidden

LAB - 3 | 5 Feb

↳ 23 cells hidden

Lab - 5 | 19 Feb

↳ 1 cell hidden

Lab 6 | 26 Feb

↳ 6 cells hidden

Lab 8 | 12 March

↳ 14 cells hidden

Lab 9 | 19 March

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
```

```
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_c
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0

conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

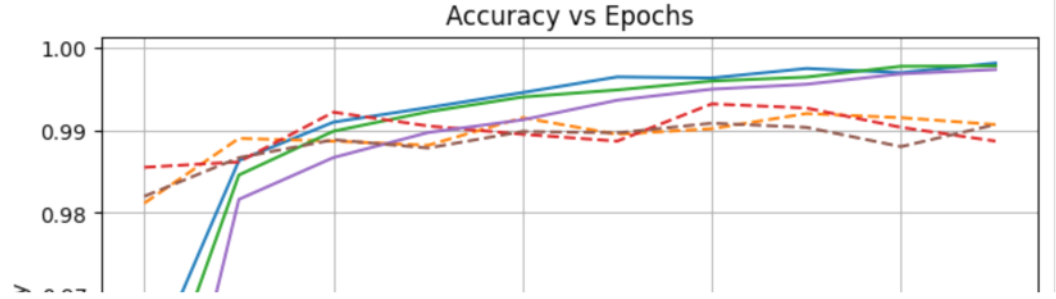
```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 57s 62ms/step - accuracy: 0.9454 - loss: 0.1774
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 80s 59ms/step - accuracy: 0.9837 - loss: 0.0516
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 58ms/step - accuracy: 0.9891 - loss: 0.0338
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9916 - loss: 0.0272
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9936 - loss: 0.0201
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 57ms/step - accuracy: 0.9956 - loss: 0.0148
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9955 - loss: 0.0136
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 57ms/step - accuracy: 0.9968 - loss: 0.0103
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 58ms/step - accuracy: 0.9973 - loss: 0.0079
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 49s 58ms/step - accuracy: 0.9972 - loss: 0.0080
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9921 - loss: 0.0278
Test accuracy: 0.9921000003814697
```

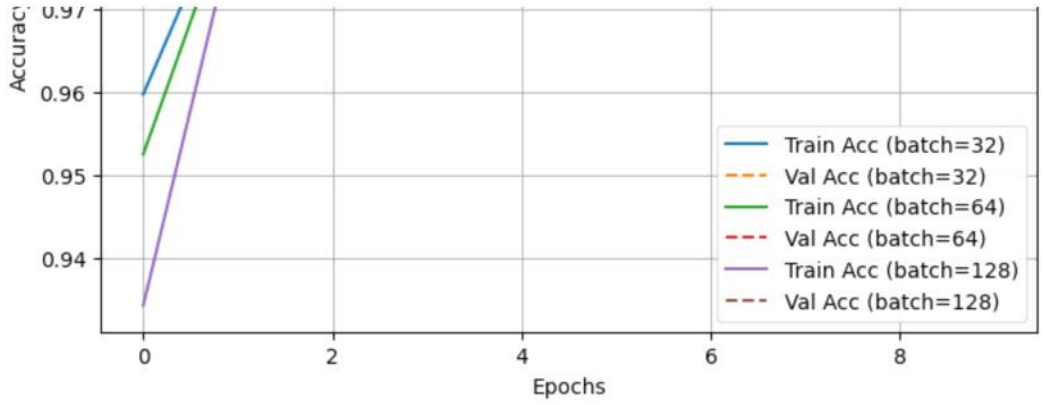
```
import matplotlib.pyplot as plt

batch_sizes = [32, 64, 128]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),
```

```
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train Acc (batch=
    plt.plot(histories[batch].history['val_accuracy'], linestyle='--', label

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Training with batch size: 32
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 64s 35ms/step - accuracy: 0.9598 - loss: 0.130
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 32ms/step - accuracy: 0.9863 - loss: 0.043
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 55s 33ms/step - accuracy: 0.9910 - loss: 0.028
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 32ms/step - accuracy: 0.9927 - loss: 0.022
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 55s 33ms/step - accuracy: 0.9945 - loss: 0.015
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 53s 31ms/step - accuracy: 0.9964 - loss: 0.011
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 55s 33ms/step - accuracy: 0.9963 - loss: 0.010
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 32ms/step - accuracy: 0.9974 - loss: 0.008
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 82s 32ms/step - accuracy: 0.9969 - loss: 0.008
Epoch 10/10
```

```
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 54s 32ms/step - accuracy: 0.9981 - loss: 0.005

Training with batch size: 64
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 51s 59ms/step - accuracy: 0.9526 - loss: 0.1615
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 58ms/step - accuracy: 0.9846 - loss: 0.0500
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 57ms/step - accuracy: 0.9899 - loss: 0.0308
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9922 - loss: 0.0258
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 57ms/step - accuracy: 0.9940 - loss: 0.0181
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 57ms/step - accuracy: 0.9949 - loss: 0.0142
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9959 - loss: 0.0114
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 48s 57ms/step - accuracy: 0.9964 - loss: 0.0098
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 57ms/step - accuracy: 0.9977 - loss: 0.0071
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 57ms/step - accuracy: 0.9978 - loss: 0.0068

Training with batch size: 128
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 47s 109ms/step - accuracy: 0.9343 - loss: 0.2260
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 108ms/step - accuracy: 0.9816 - loss: 0.0599
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 45s 107ms/step - accuracy: 0.9867 - loss: 0.0415
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 108ms/step - accuracy: 0.9897 - loss: 0.0321
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 108ms/step - accuracy: 0.9912 - loss: 0.0259
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 46s 109ms/step - accuracy: 0.9936 - loss: 0.0203
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 106ms/step - accuracy: 0.9949 - loss: 0.0159
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 83s 108ms/step - accuracy: 0.9955 - loss: 0.0135
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 82s 108ms/step - accuracy: 0.9968 - loss: 0.0104
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 46s 108ms/step - accuracy: 0.9973 - loss: 0.0080
```

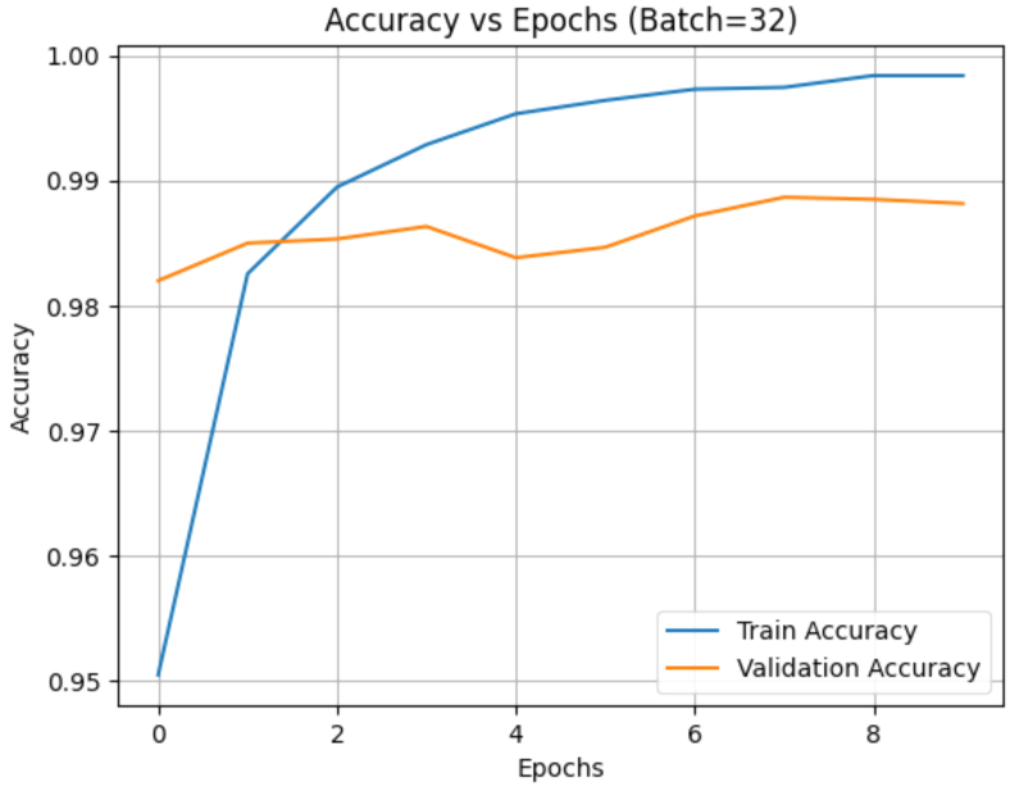





```
import matplotlib.pyplot as plt

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.1)

# Plot accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy vs Epochs (Batch=32)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 47s 27ms/step - accuracy: 0.9505 - loss: 0.164
Epoch 2/10
```

```
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 76s 23ms/step - accuracy: 0.9826 - loss: 0.053
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 39s 23ms/step - accuracy: 0.9895 - loss: 0.032
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 43s 24ms/step - accuracy: 0.9929 - loss: 0.021
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 38s 22ms/step - accuracy: 0.9953 - loss: 0.015
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 24ms/step - accuracy: 0.9964 - loss: 0.011
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 38s 22ms/step - accuracy: 0.9973 - loss: 0.008
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 40s 24ms/step - accuracy: 0.9974 - loss: 0.006
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 37s 22ms/step - accuracy: 0.9984 - loss: 0.005
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 40s 24ms/step - accuracy: 0.9984 - loss: 0.005
```




```
import matplotlib.pyplot as plt

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(10, activation='softmax')
])
```

```
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.1)

# Plot accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy vs Epochs (Batch=32)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 29s 16ms/step - accuracy: 0.9343 - loss: 0.235
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 26s 15ms/step - accuracy: 0.9754 - loss: 0.085
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 27s 16ms/step - accuracy: 0.9818 - loss: 0.062
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 27s 16ms/step - accuracy: 0.9847 - loss: 0.051
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 38s 14ms/step - accuracy: 0.9869 - loss: 0.042
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 17ms/step - accuracy: 0.9886 - loss: 0.035
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 27s 16ms/step - accuracy: 0.9904 - loss: 0.030
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 16ms/step - accuracy: 0.9921 - loss: 0.026
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 38s 15ms/step - accuracy: 0.9936 - loss: 0.021
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 27s 16ms/step - accuracy: 0.9947 - loss: 0.018
```